



The (LLM) Architects

ARC-AGI 2024 Winning Solution

Jan Disselhoff, Daniel Franzen, David Hartmann

GTC 25

Winning Solution of the ARC AGI Challenge

What is ARC

Reasoning as Benchmark
Task Examples
ARC is hard!

Key Progresses

DSL for ARC Problems
Additional Datasets
Test-Time Training

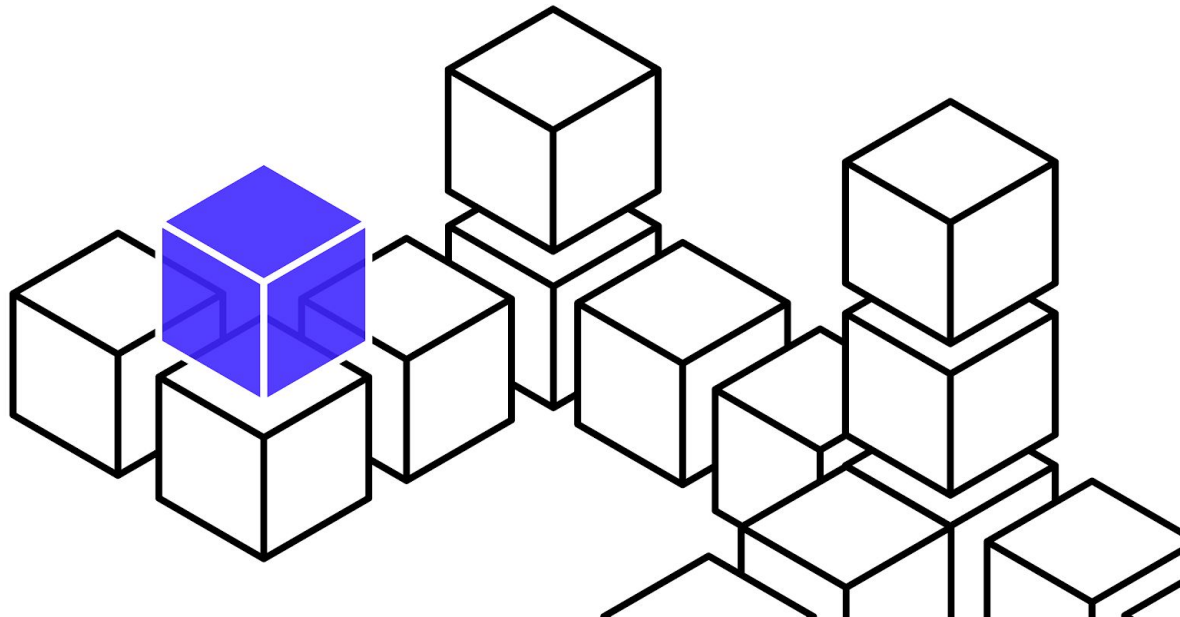
Optimizations

Challenge Constraints
Tokenization
VRAM Reduction Tricks

Our Pipeline

Efficient Fine-Tuning
DFS Inference
Candidate Ranking

What is ARC-AGI ?

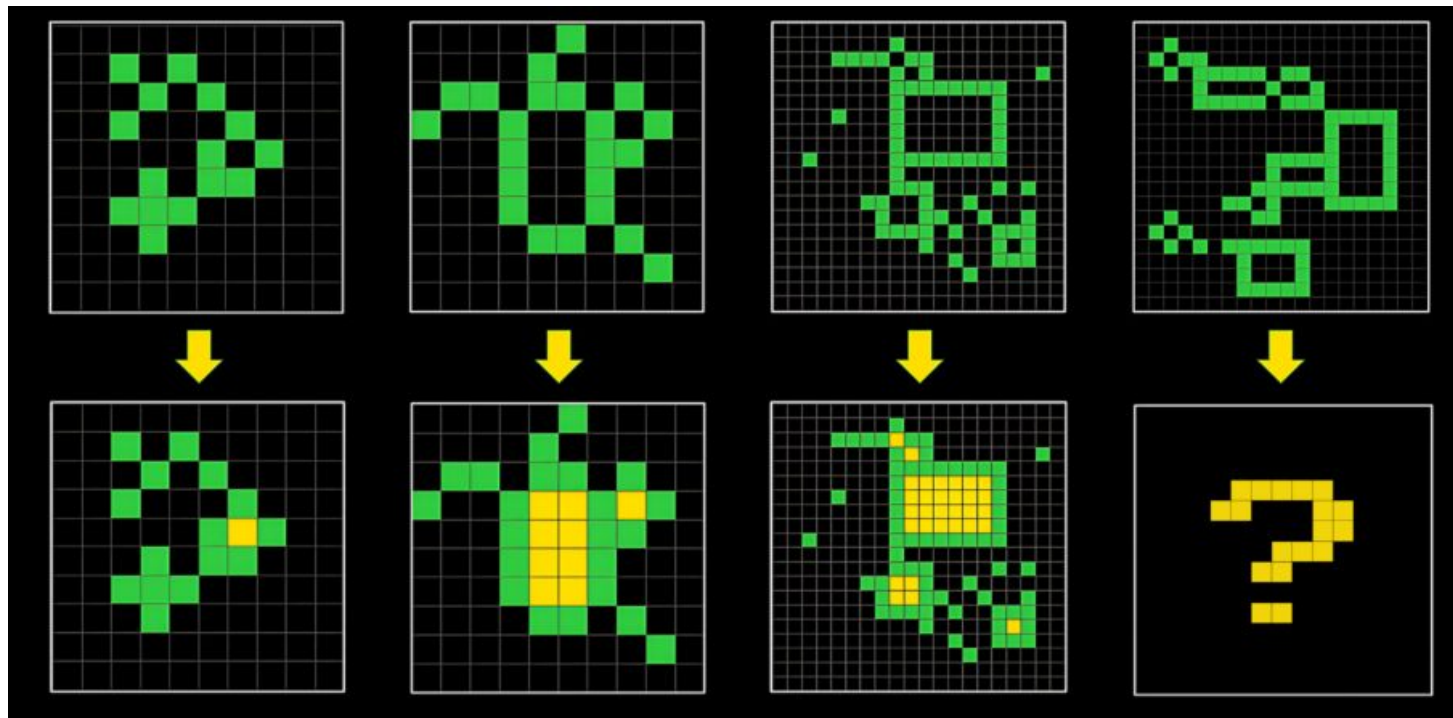


**“It’s easy for humans,
but hard for AI.”**

– arcprize.org

■ What is ARC-AGI?

5



ARC-AGI = Reasoning As A Benchmark

A series of cognitive challenges on small grids

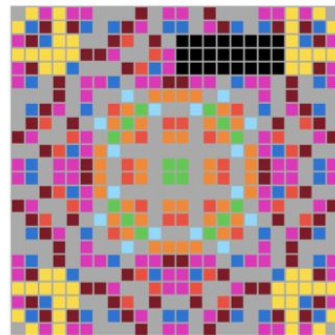
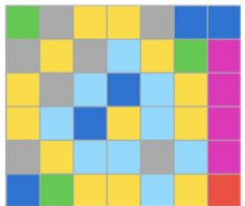
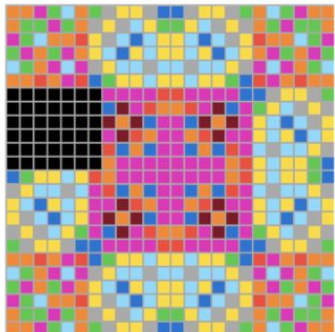
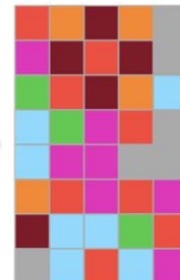
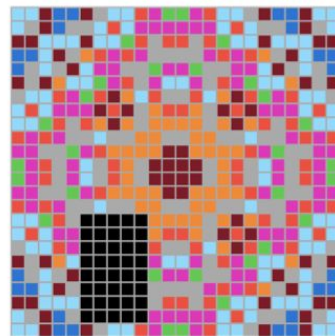
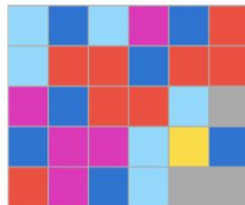
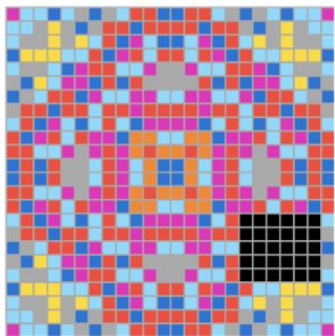
Rules must be learned from only a few (2 – 6) examples

Simple for humans, very hard for AI

- + "explicitly designed to compare artificial intelligence with human intelligence"
-

■ What is ARC-AGI?

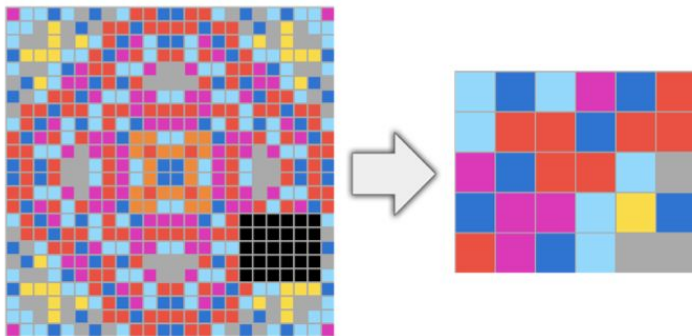
7



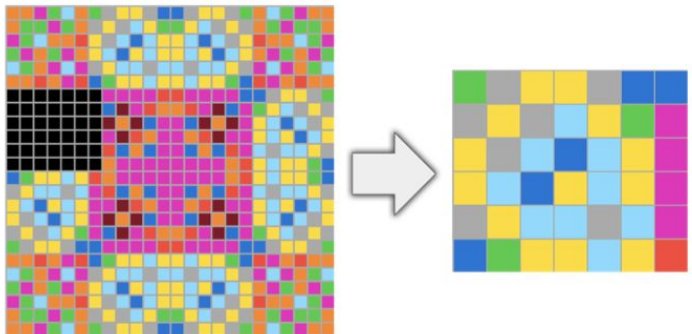
?

■ What is ARC-AGI?

8



Only Pixel-Perfect Predictions Count!
(... and LLMs don't "see" 2D data)

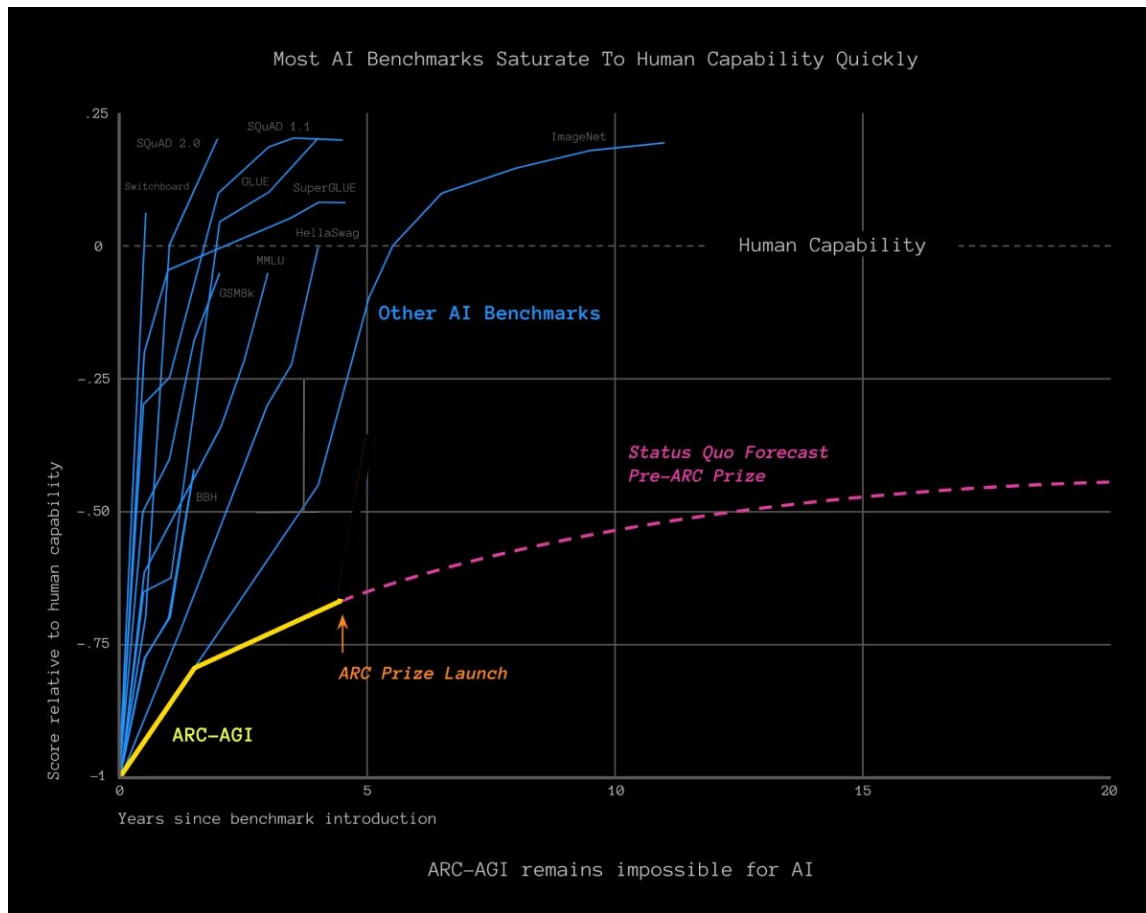


How well do frontier LLMs perform?

2024 ARC-AGI-PUB HIGH SCORES				
o1-preview	18%	21%	CODE	
Claude 3.5 Sonnet	14%	21%	CODE	
GPT-4o	5%	9%	CODE	
Gemini 1.5	4.5%	8%	CODE	
Score (semi-private eval) / Score (public eval)				

What is ARC-AGI?

Claim:
"Progress has stalled"

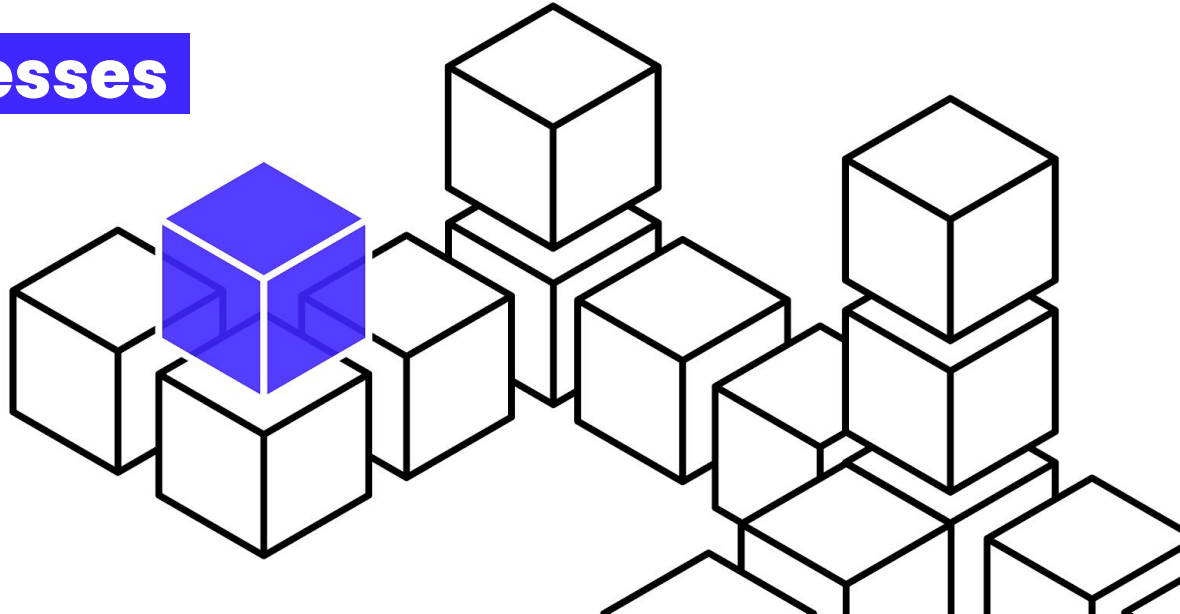


How well do Open-Source LLMs perform?

The screenshot shows the Kaggle website interface for the ARC Prize 2024 competition. On the left is a sidebar with navigation links: Create, Home, Competitions (highlighted), Datasets, Models, Code, Discussions, Learn, and More. The main content area features a search bar at the top, followed by the competition title 'ARC Prize 2024' and a subtitle 'ABSTRACTION AND REASONING CORPUS · FEATURED CODE COMPETITION · 3 MONTHS AGO'. Below this is a description: 'Create an AI capable of solving reasoning tasks it has never seen before'. A grid of small images representing various reasoning tasks is shown on the right. The 'Leaderboard' tab is selected, displaying a search bar for the leaderboard and a note: 'The private leaderboard is calculated over the same rows as the public leaderboard in this competition. This competition has completed. This leaderboard reflects the final standings.' Buttons for 'Raw Data' and 'Refresh' are also visible.



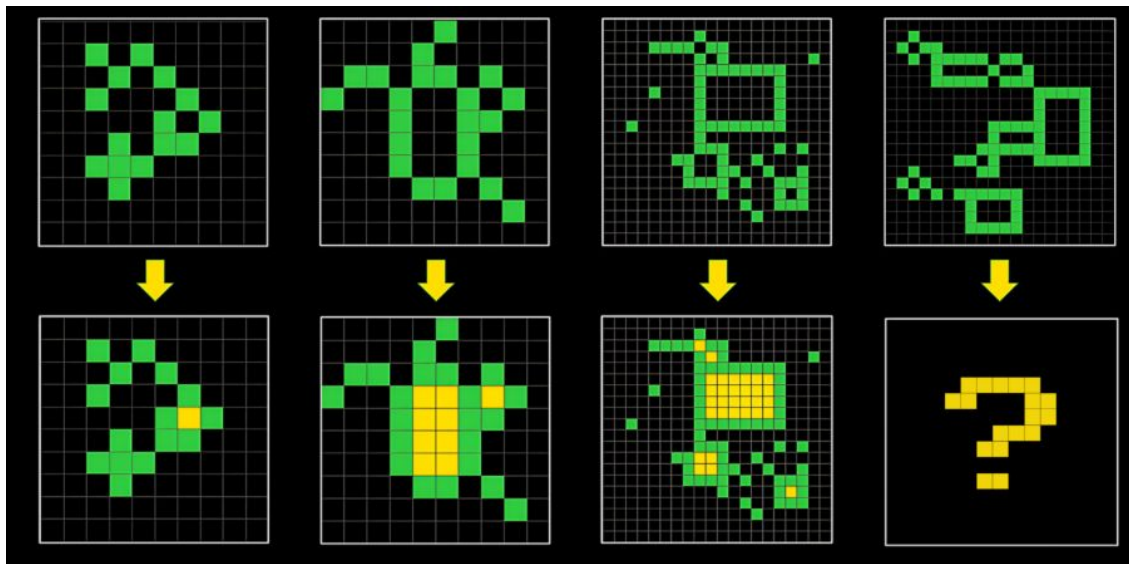
Community **Progresses**



Community Identified:
Test-Time Training,
DSL of Tasks,
Data Augmentation

Test-Time Training

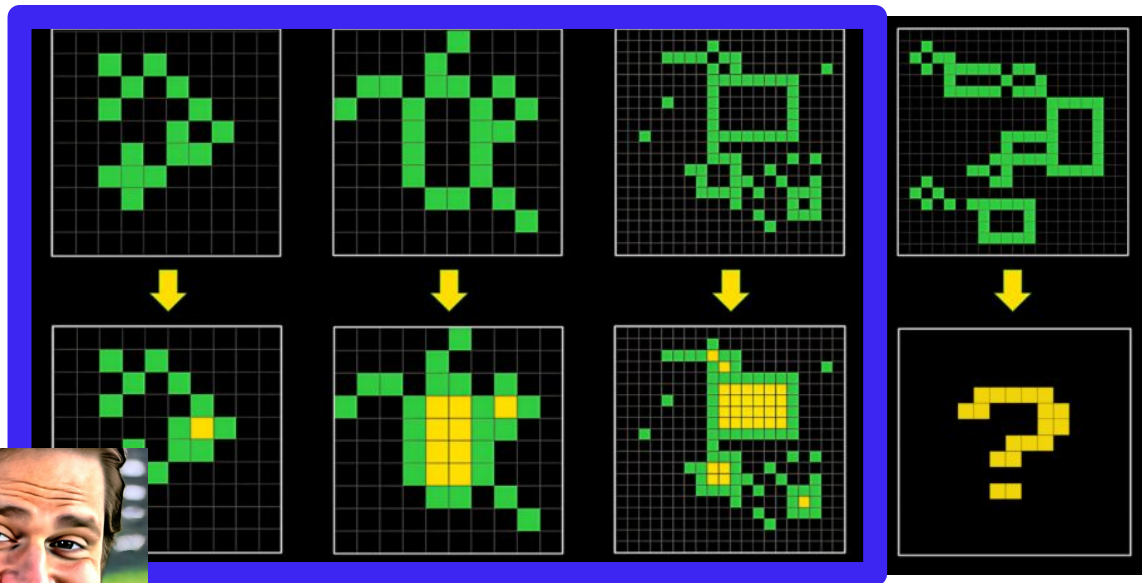
- Arc has small conceptual Tasks
- Each task includes some examples
- ... even the test examples
- **Idea:** Use the examples as a small Dataset



Test-Time Training

- Arc has small conceptual Tasks
- Each task includes some examples
- ... even the test examples
- **Idea:** Use the examples as a small Dataset

Free Additional
Training Data



Domain-Specific Language of ARC Tasks

Introduced by github.com/michaelhodel/re-arc

- Functions designed explicitly to manipulate ARC-like tasks
- In previous challenges: combination with program search to solve arc tasks
- In the 2024 challenge: program generation using LLMs

```
def generate_91413438(diff_lb: float, diff_ub: float) -> dict:
    cols = interval(1, 10, 1)
    h = unifint(diff_lb, diff_ub, (2, 5))
    w = unifint(diff_lb, diff_ub, (2, 5))
    maxnb = min(h * w - 1, min(30//h, 30//w))
    minnb = int(0.5 * ((4 * h * w + 1) ** 0.5 - 1)) + 1
    nbi = unifint(diff_lb, diff_ub, (0, maxnb - minnb))
    nb = min(max(minnb, maxnb - nbi), maxnb)
    fgc = choice(cols)
    c = canvas(0, (h, w))
    obj = sample(totuple(asindices(c)), h * w - nb)
    gi = fill(c, fgc, obj)
    go = canvas(0, (h * nb, w * nb))
    for j in range(h * w - nb):
        loc = (j // nb, j % nb)
        go = fill(go, fgc, shift(obj, multiply((h, w), loc)))
    return {'input': gi, 'output': go}
```


Domain-Specific Language of ARC Tasks

Introduced by github.com/michaelhodel/re-arc

Datasets generated with DSLs

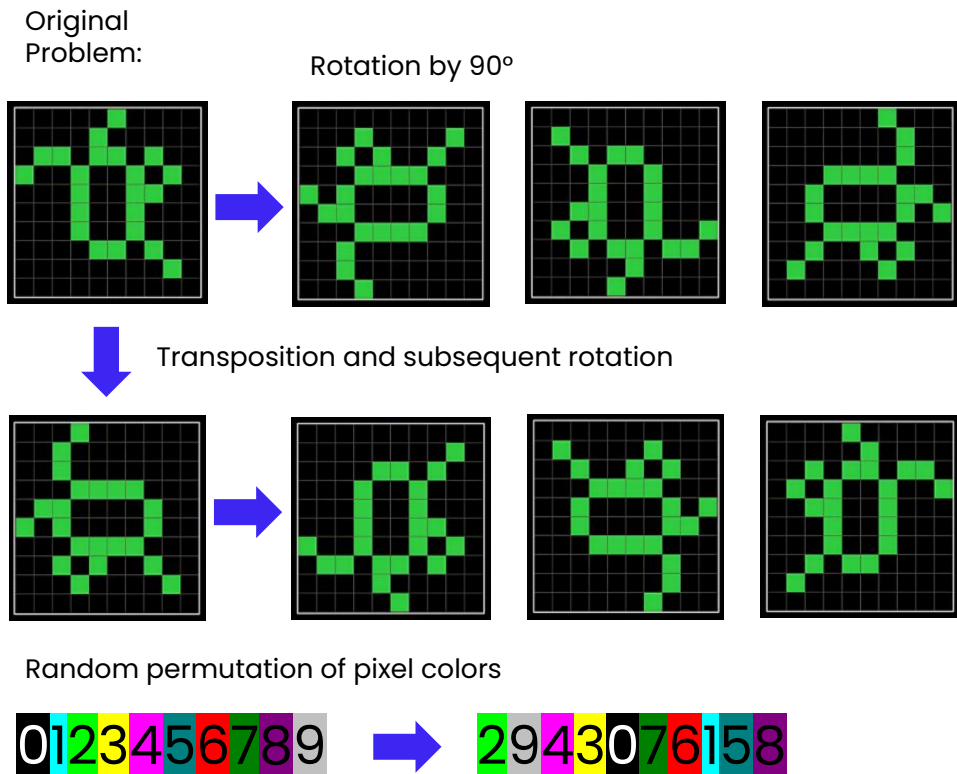
- Re-ARC
 - Reimplements original ARC challenges
 - (Only 400 of the 800 public available tasks)
- ARC-Heavy
 - LLM-based generators

```
def generate_91413438(diff_lb: float, diff_ub: float) -> dict:
    cols = interval(1, 10, 1)
    h = unifint(diff_lb, diff_ub, (2, 5))
    w = unifint(diff_lb, diff_ub, (2, 5))
    maxnb = min(h * w - 1, min(30//h, 30//w))
    minnb = int(0.5 * ((4 * h * w + 1) ** 0.5 - 1)) + 1
    nbi = unifint(diff_lb, diff_ub, (0, maxnb - minnb))
    nb = min(max(minnb, maxnb - nbi), maxnb)
    fgc = choice(cols)
    c = canvas(0, (h, w))
    obj = sample(totuple(asindices(c)), h * w - nb)
    gi = fill(c, fgc, obj)
    go = canvas(0, (h * nb, w * nb))
    for j in range(h * w - nb):
        loc = (j // nb, j % nb)
        go = fill(go, fgc, shift(obj, multiply((h, w), loc)))
    return {'input': gi, 'output': go}
```

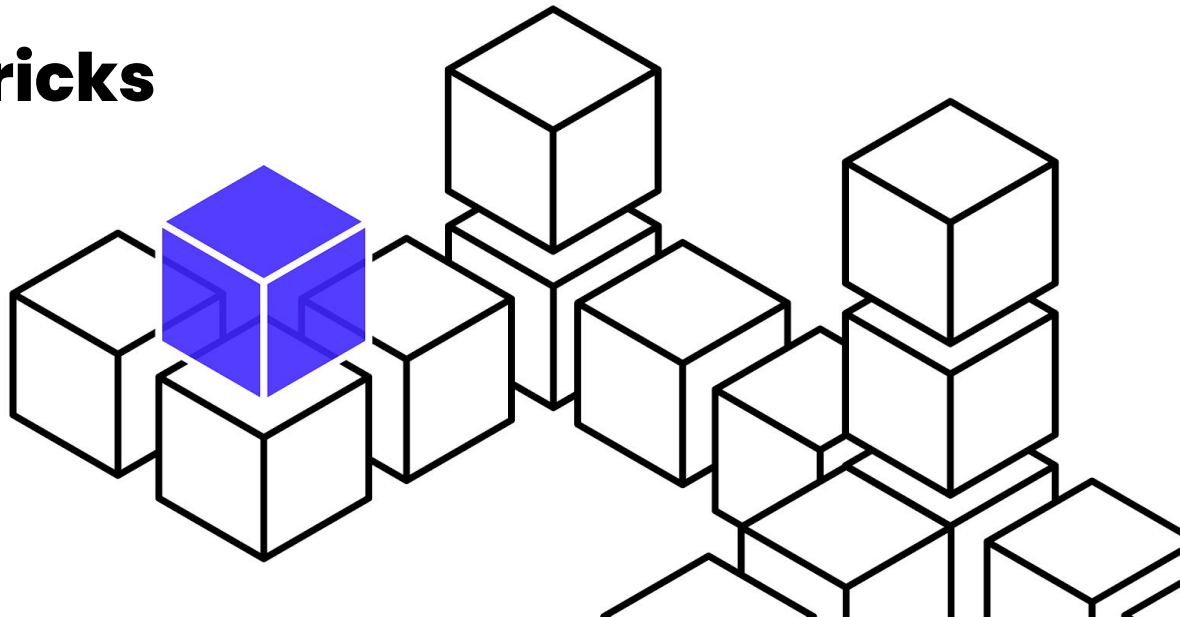
■ Key Progresses

Data Augmentation

- Modifications under which the problem is still solvable
- "Asking the same question from different perspectives"
- Can be used in every step
 - Training: increase amount of training data
 - Inference: increase chance of sampling the correct solution



Optimizations & Tricks



Efficient Use of Limited Resources (2 x NVIDIA T4 GPU)

Why Are Resources Limited

Kaggle challenge limitations

- “Code challenge” – code must be uploaded and is executed by kaggle
- 2x NVIDIA T4 GPUs (with 16GB VRAM each)
- 4 CPU cores
- max. 12 hours runtime



Two Steps:

- Offline and extensive Fine-Tune on NVIDIA H100 GPU
- Second “Test-Time” Fine-Tune on challenge examples on 2 NVIDIA T4 GPUs (to utilize both GPUs, we split the dataset in two parts)

■ Key Progresses

Tokenizer Changes

- Text is split into chunks
- Makes inference cheaper
- But number problems become harder
- This is bad for us!

We remove 99.95% of all Tokens and merges from Model!



Text is split into chunks of differing length.
Most words are single tokens... but not all!
Strawberry is a good example.

Numbers are often tokenized into triplets:
154864616789451984651



[1199, 374, 6859, 1139, 27855, 315, 61469, 3160, 627, 13622, 4339, 527, 3254, 11460, 1975, 719, 539, 682, 4999, 2645, 675, 15717, 374, 264, 1695, 3187, 382, 28336, 527, 3629, 4037, 1534, 1139, 24657, 2641, 512, 10559, 19355, 21379, 16474, 20360, 23812, 23409]

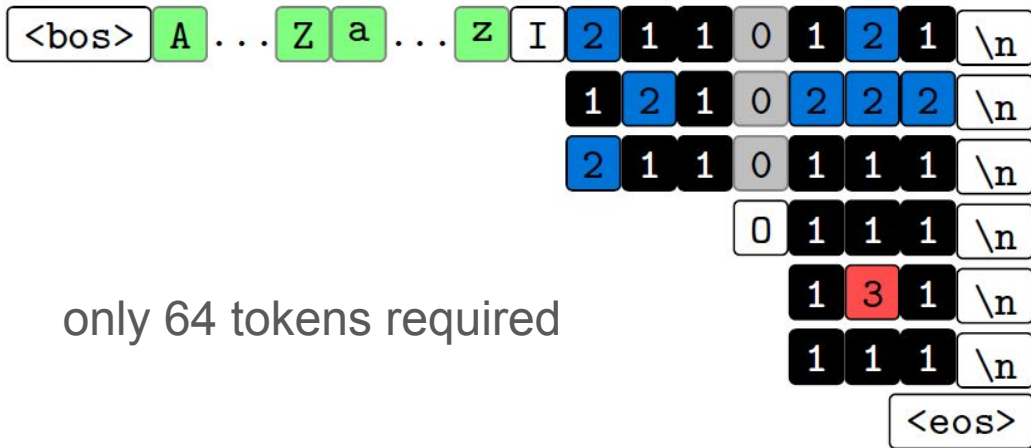
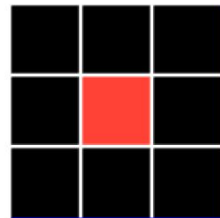
- Model is smaller ✓
- Inference is simpler ✓
- No hallucinations ✓
- No filler tokens ✓

Takeaway

When we have very specific use cases it can be useful to **reduce** LLM capabilities

Our Tokenization approach

- Grids are tokenized line-by-line
- End-of-line indicates start of new line
- I and O mark beginning of input and output
- Colors are represented by number tokens
- 2D positional encoding not necessary

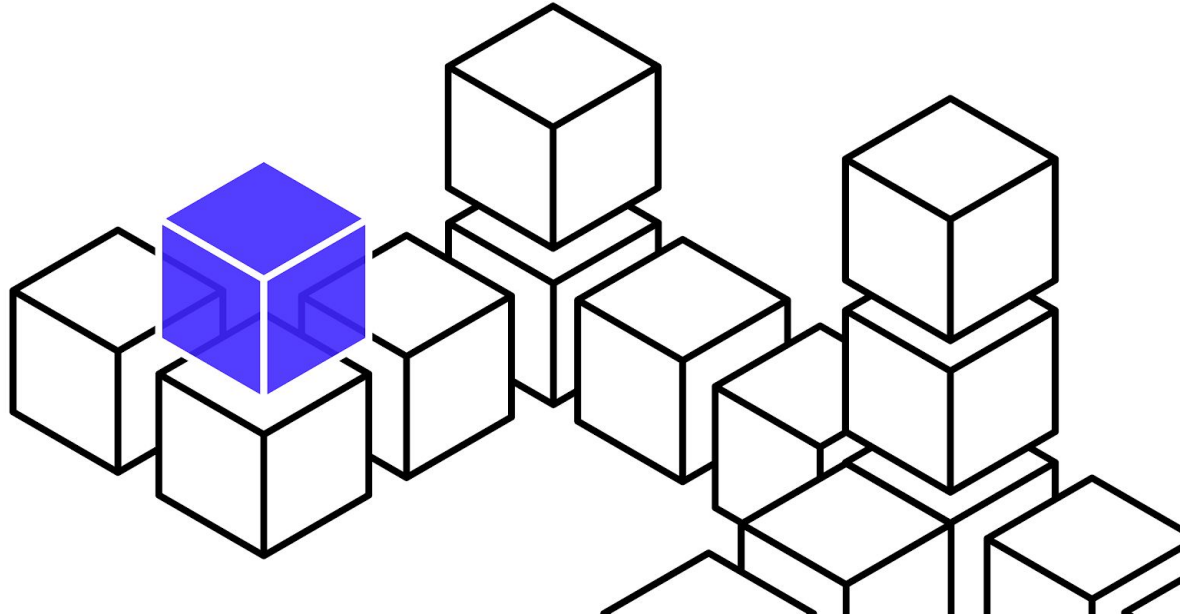


only 64 tokens required

VRAM Reduction Techniques

- Shrinking embedding layer from 128k to only **64 tokens**
- **4bit** quantized base model
- **QLoRA** finetuning
- **Gradient Checkpointing**
- using the **unsloth** library

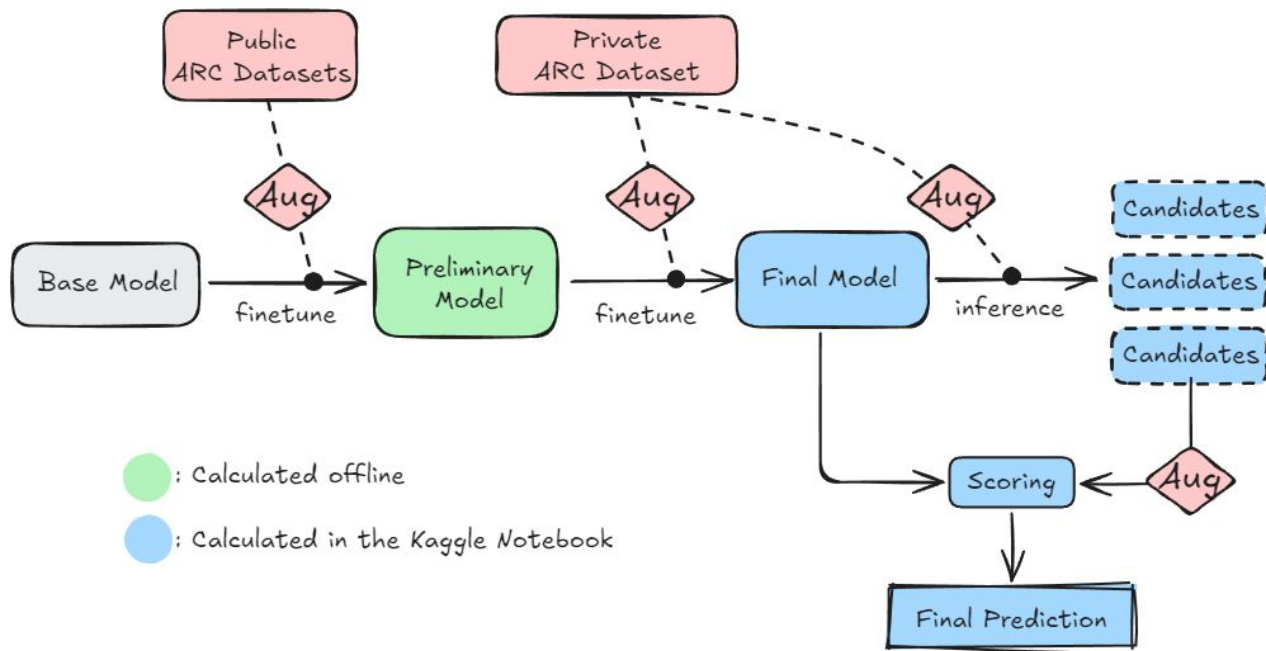
Our Pipeline



No prompt tuning
No chain-of-thought
No large model

■ Our Pipeline

27



Finetune on Augmented Dataset

- Create Augmented Dataset from different sources
 - ReARC, ARC-Heavy, ConceptArc...
- Finetune base LLM using QLoRA and unsloth
- “Small” Language Model:
NVIDIA Mistral-NeMo-Minitron-8B-Base
- Trained for ~100h on NVIDIA H100 GPU

DFS based Inference

- Trained model is good but how to sample?
- Standard Multinomial Sampling has issues:
 - Repetition of same solution
 - No guarantees of sampling probability
 - Increasing chance of errors for longer problems
- Beam search slightly better, but at cost of additional VRAM
- Is there an alternative?

Yes! Write your own sampling algorithm!

DFS based Inference

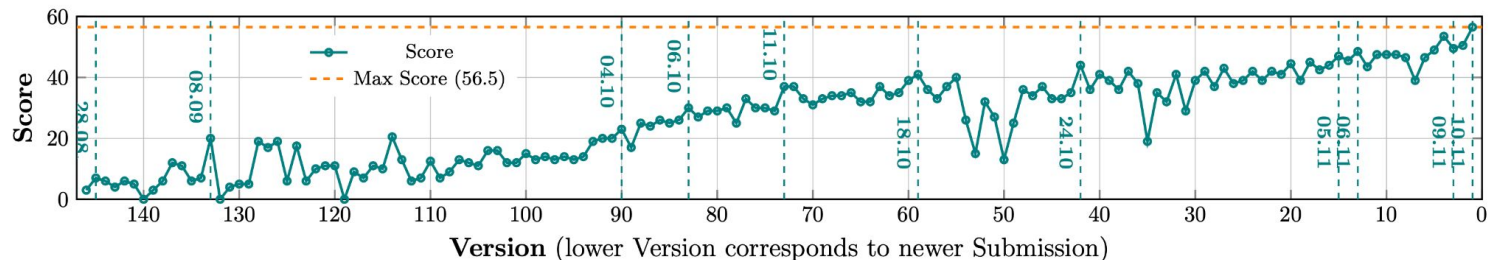
- DFS based sampling with threshold
- Search through partial solutions in Tree, but discard all paths below some threshold
- Can use inference caches to make this fast and cheap!
 - Same VRAM as standard sampling
- NO Repetitions!
- NO low probability solutions!

Augmented Scoring

- LLMs have causal inference
- But ARC-AGI tasks are 2D!
- Some problems are easier to solve from certain perspectives
- Can we use this?

Augmented Scoring

- Take solution Candidates
- Calculate sampling probability from each perspective
- Aggregate using mean of log-probs



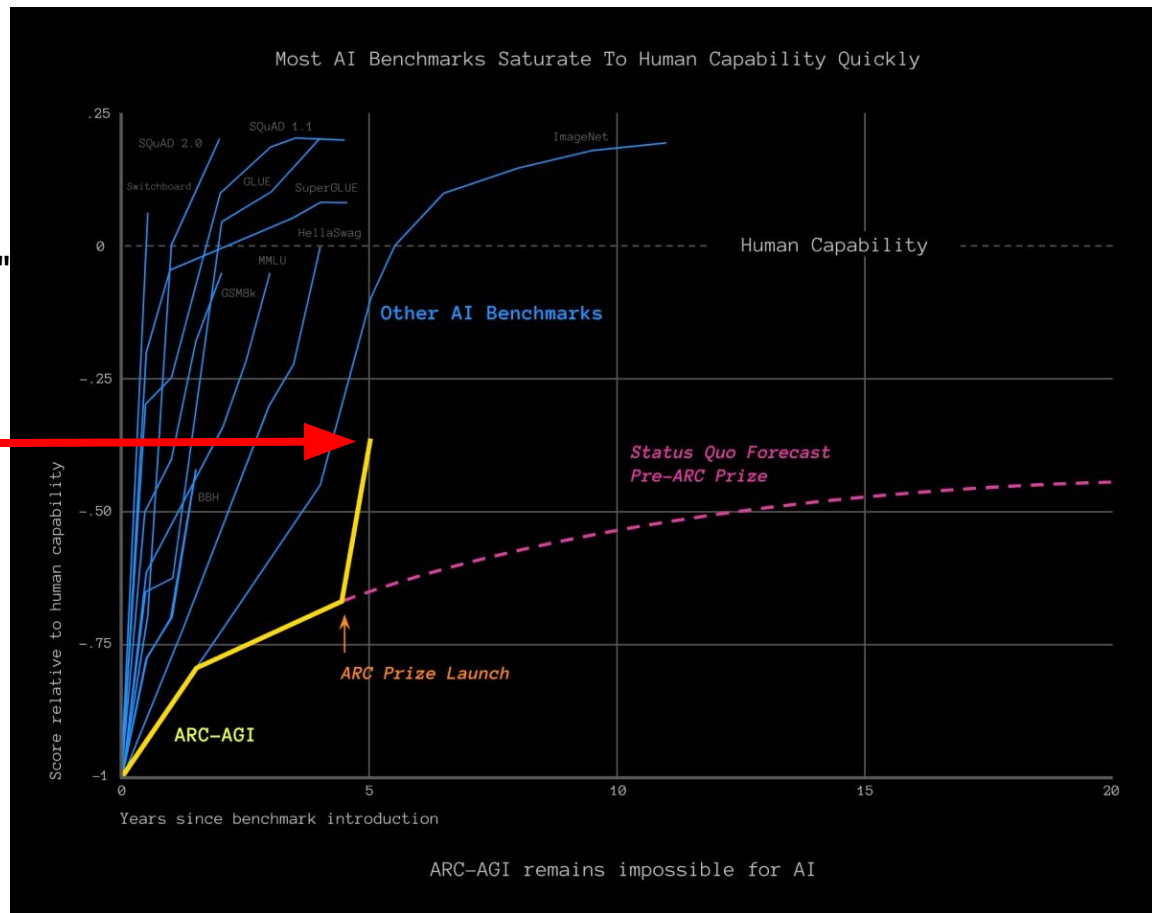
Date	Score	Significant Changes	Model	Train Examples	Aug.	Sampling Strategy	Selecion Strategy
28.08.2024	7.0	Inference only (normal & transpose)	Mistral-Nemo-12B	26 k	2x	stochastic	-
08.09.2024	20.0	Test-time retraining	Mistral-Nemo-12B	26 k	2x	stochastic	-
04.10.2024	23.0	Model switch + ReArc + similarity_select	Llama-3.2-3B (uncensored)	74 k	4x	stochastic	similarity
06.10.2024	30.0	16-fold inference	Llama-3.2-3B (uncensored)	74 k	16x	greedy	similarity
11.10.2024	37.0	Calculating scores with augmentations	Llama-3.2-3B (uncensored)	74 k	16x	greedy	4xAugScore
18.10.2024	41.0	Longer pre-training	Llama-3.2-3B (uncensored)	166 k	16x	stochastic	4xAugScore
24.10.2024	44.0	DFS inference	Llama-3.2-3B (uncensored)	166 k	16x	DFS 10%	4xAugScore
05.11.2024	47.0	Model switch + add ConceptArc	NeMo-Minitron-8B	178 k	8x	DFS 17%	8xAugScore
06.11.2024	48.5	Longer pre-train + include full eval set	NeMo-Minitron-8B "Nemo-full"	368 k	8x	DFS 17%	8xAugScore
09.11.2024	53.5	16-fold inference	NeMo-Minitron-8B "Nemo-full"	368 k	16x	DFS 14%	8xAugScore
10.11.2024	56.5*	Adding ARC-Heavy (for pre-train)	NeMo-Minitron-8B "Nemo-heavy"	531 k	16x	DFS 20%	8xAugScore

What is ARC-AGI?

Claim:

"Progress has stalled"

Our Solution



In the meantime ...

How well do frontier LLMs perform?

2024 ARC-AGI-PUB HIGH SCORES				
o1-preview	18%	21%	CODE	
Claude 3.5 Sonnet	14%	21%	CODE	
GPT-4o	5%	9%	CODE	
Gemini 1.5	4.5%	8%	CODE	
Score (semi-private eval) / Score (public eval)				

How well do frontier LLMs perform?

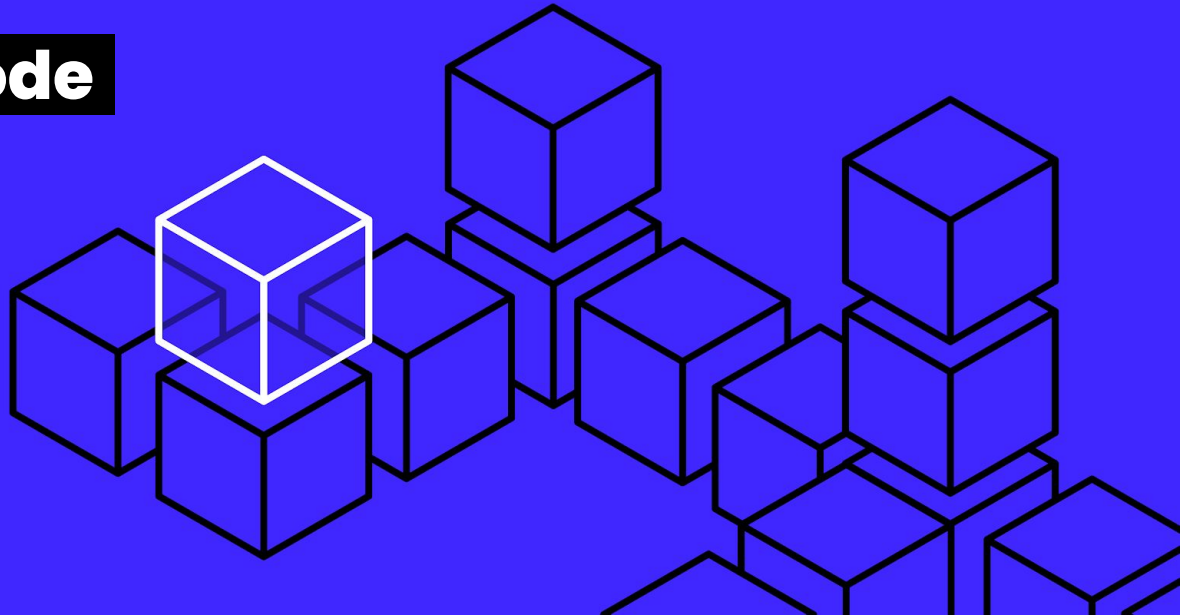
2024 ARC-AGI-PUB HIGH SCORES			
o3 (coming soon)	75.7%	62.6%	?
o1-preview	16%	21%	CODE
Claude 3.5 Sonnet	14%	21%	CODE
GPT-4o	5%	9%	CODE
Gemini 1.5	4.5%	8%	CODE
Score (semi-private eval) / Score (public eval)			

- LLMs **can** be used for cognitive tasks
 - **IF** they are finetuned well
- Restricting the problem space can be very effective
 - **IF** the problem allows it
- Sampling correctly is key!
- Useful trick: Augment queries to double check results

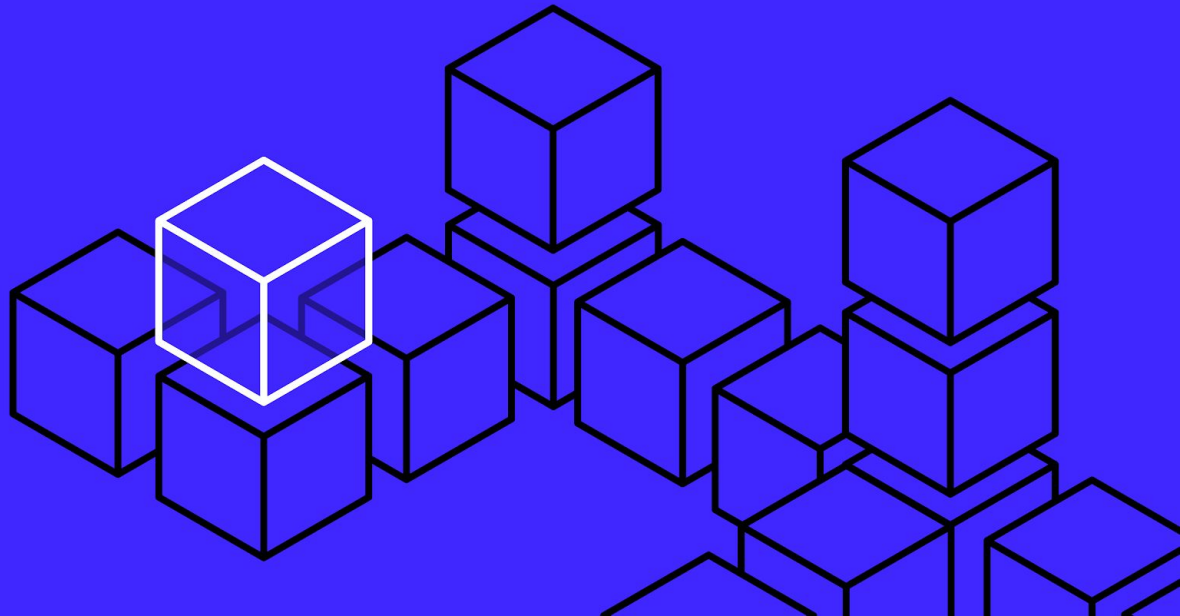
o3 price per task: 17\$
Ours per task: 0.14\$*

* assuming 2.5\$ per hour for a NVIDIA H100 GPU

Tutorial **Use Our Code**



Frequently Asked Questions



**Thank You For
Listening!
Questions**

